MITRE-CORE HGNN Architecture Deep Dive

Table of Contents

High-Level Overview

The core model is `MITREHeteroGNN`, a PyTorch Geometric-based architecture designed to correlate security alerts across multiple domains (network, endpoint, IIoT) into actionable multi-stage attack campaigns.

It uses a **Heterogeneous Graph** approach because cybersecurity data naturally consists of different entity types (IPs, users, hosts, alerts) with distinct feature spaces and relationships, which standard homogeneous GNNs cannot expressively capture.

Neural Network Architecture

Layer Count & Structure

The model uses **2 main GNN layers** (`num_layers=2`) by default:

- **Input Layer**: 9 separate Linear encoders (one per node type)
- **Hidden Layers**: 2× GATConv layers with residual connections
- **Output Layer**: MLP classification head (2 Linear layers)

Architecture Diagram

```
Input Features (various dimensions)
    ↓
[Linear Encoders] → 128-dim unified space
    ↓
[GATConv Layer 1] + Residual + LayerNorm
    ↓
[GATConv Layer 2] + Residual + LayerNorm
    ↓
[MLP Classification Head] → Cluster logits
```

Layer-by-Layer Breakdown

Input Encoders (Lines 161-169)

```
self.alert_encoder = Linear(-1, hidden_dim)  # 64 -> 128
self.user_encoder = Linear(-1, hidden_dim)    # 32 -> 128
# ... 7 more encoders
```

Technical Details:

- **Why `Linear(-1, hidden_dim)`?**: `-1` allows dynamic input dimensions, crucial for heterogeneous data where feature dimensions vary per node type
- **Purpose**: Projects all node types into a unified 128-dimensional latent space
- **No activation here**: Pure linear projection to preserve raw feature information

GATConv Layers (Lines 179-210)

```
conv_dict[("alert", rel, "alert")] = GATConv(
    hidden_dim, hidden_dim // num_heads,  # 128 -> 32 per head
    heads=num_heads, dropout=dropout, add_self_loops=False,
)
```

Technical Specifications:

- **Multi-head attention**: 4 heads × 32 dimensions = 128 total
- `add_self_loops=False`: Prevents nodes from attending to themselves, forcing reliance on graph structure
- **Why GAT over GCN?**: Attention weights provide interpretability for security analysts

Residual Connections (Lines 306-311)

```
# Critical sequence: Conv -> ReLU -> Dropout -> LayerNorm -> Residual
v = F.relu(v)                    # Non-linearity
v = F.dropout(v, p=0.3, training=self.training)  # Regularization
v = norm(v)                      # Normalization
x_dict[k] = v + residual         # Skip connection
```

Why this order?

1. **ReLU first**: Introduces non-linearity before regularization
2. **Dropout after ReLU**: Prevents co-adaptation of activated features
3. **LayerNorm last**: Normalizes final representations before residual addition

---

Feature Engineering Pipeline

Alert Feature Encoding (Lines 525-574)

The `_encode_alert_features` function creates a **6-dimensional feature vector**:

```
features = np.column_stack([
    tactics,        # MITRE tactic (categorical -> integer)
    alert_types,    # Binary: attack vs normal
    hour / 23.0,    # Normalized time (0-1)
    dow / 6.0,      # Day of week (0-1)
    protocols,      # Network protocol (categorical)
    services        # Service type (categorical)
])
```

Temporal Feature Engineering

```
# Handles both Unix timestamps and datetime strings
if pd.api.types.is_numeric_dtype(ts_values):
    dates = pd.to_datetime(ts_values, unit='s', errors='coerce')
else:
    dates = pd.to_datetime(ts_values, errors='coerce')

hour = np.nan_to_num(dates.dt.hour.values, nan=0.0) / 23.0
dow = np.nan_to_num(dates.dt.dayofweek.values, nan=0.0) / 6.0
```

Technical Details:

- **Normalization**: Divides by max possible value for [0,1] range
- **Robustness**: `nan_to_num` handles missing timestamps
- **Cyclical encoding**: Could be enhanced with sin/cos for temporal continuity

---

Message Passing Flow

Forward Pass Sequence (Lines 267-318)

Step 1: Node Encoding

```
x_dict[ntype] = encoder(data[ntype].x)  # Project to 128-dim
```

Step 2: Edge Filtering

```
available_edges = {
    et: ei for et, ei in data.edge_index_dict.items()
    if et[0] in x_dict and et[2] in x_dict  # Only use existing node types
}
```

Step 3: Layer-wise Processing

```
for i, conv in enumerate(self.convs):
    x_dict_new = conv(x_dict, conv_edges)  # Message passing
    # Apply: ReLU -> Dropout -> LayerNorm -> Residual
```

Step 4: Classification

```
if self.domain_heads is not None and domain is not None:
    cluster_logits = self.domain_heads[domain](alert_embeddings)
else:
    cluster_logits = self.cluster_classifier(alert_embeddings)
```

Graph Construction Details

Edge Types & Their Semantics

Alert-to-Alert Correlations

```
# Shared IP addresses (potential lateral movement)
for i, ai in enumerate(idxs):
    for aj in idxs[i + 1:]:
        add_edge(("alert", "shares_ip", "alert"), ai, aj)
```

Temporal Proximity

```
# Connect alerts within temporal window
for j in range(i + 1, min(i + 100, len(aidxs))):
    diff_h = abs((ts.iloc[j] - tsi).total_seconds() / 3600)
    if diff_h <= self.temporal_window:  # Default 1 hour
        add_edge(("alert", "temporal_near", "alert"), ai, aidxs[j])
```

Cross-Sensor Bridge Edges

```
# Critical for multi-domain correlation
ip_to_host: Dict[str, str] = {}  # Mine IP->hostname mapping
for ip_val, host_val in ip_to_host.items():
    add_edge(("ip", "resolves_to", "host"), iid, hid)
```

Backbone Architecture & Freezing

What is a Backbone?

In deep learning, a **backbone** refers to the core feature extraction layers of a neural network that learn rich, hierarchical representations from raw input data. In MITRE-CORE's HGNN:

- **Backbone Components**: Input encoders + GATConv layers + residual connections
- **Backbone Output**: 128-dimensional node embeddings that capture structural and semantic patterns
- **Classification Head**: Separate MLP that maps embeddings to cluster assignments

Code Structure: Backbone vs Head

```
# === BACKBONE (Feature Extraction) ===
class MITREHeteroGNN(nn.Module):
    def __init__(self):
        # Input encoders
        self.alert_encoder = Linear(-1, hidden_dim)
        # ... other encoders

        # GNN layers (core backbone)
        self.convs = nn.ModuleList([...])  # GATConv layers
        self.layer_norms = nn.ModuleList([...])

    def get_backbone_embeddings(self, data):
        """Extract embeddings BEFORE classification head"""
        # Message passing through GNN layers
        # Returns: x_dict with 128-dim embeddings per node type

    # === CLASSIFICATION HEAD ===
    self.cluster_classifier = nn.Sequential(
        nn.Linear(hidden_dim, hidden_dim // 2),  # 128 -> 64
        nn.ReLU(),
        nn.Dropout(dropout),
        nn.Linear(hidden_dim // 2, num_clusters), # 64 -> 10
    )
```

Why Freeze the Backbone?

Freezing the backbone is a critical technique in transfer learning with several important benefits:

1. Preserve Learned Representations

```
# Freeze backbone parameters
for param in model.backbone.parameters():
    param.requires_grad = False
```

```
# Only train classification head
for param in model.cluster_classifier.parameters():
    param.requires_grad = True
```

Why?

- The backbone has learned universal patterns of alert correlation across domains
- These representations are valuable and shouldn't be corrupted by domain-specific fine-tuning
- Prevents catastrophic forgetting of cross-domain knowledge

2. Computational Efficiency

- **Reduced memory usage**: Only gradients for classifier head need storage
- **Faster training**: Fewer parameters to optimize
- **Lower risk of overfitting**: Smaller parameter space for limited target domain data

3. Domain Adaptation Strategy

```
# Multi-domain training with shared backbone
model = MITREHeteroGNN(domain_cluster_dims={
    'beth': 2,     # Binary classification
    'unsw': 10,    # Multi-class
    'optc': 2      # Binary classification
})

# Backbone learns universal patterns
# Domain heads learn dataset-specific decision boundaries
```

4. Practical Use Cases in MITRE-CORE

Scenario 1: New Dataset Integration

```
# Load pre-trained backbone from UNSW+BETH+OpTC
checkpoint = torch.load('multidomain_v2/best_supervised.pt')
model.load_state_dict(checkpoint)

# Freeze backbone, train only on new domain
freeze_backbone(model)
train_on_new_dataset(model, new_domain_data)
```

Scenario 2: Rapid Deployment

```
# Pre-compute backbone embeddings
embeddings = model.get_backbone_embeddings(alert_graph)
# Store embeddings for fast real-time clustering
```

5. When to Unfreeze

```
# Progressive unfreezing for better adaptation
def progressive_unfreeze(model, num_layers=1):
    """Unfreeze last N GNN layers for fine-tuning"""
    for i, conv in enumerate(model.convs[-num_layers:]):
        for param in conv.parameters():
            param.requires_grad = True
```

Unfreeze when:

- Target domain is very different from training domains
- Large amount of labeled data available
- Backbone features don't capture target domain specifics

Activation Functions & Their Rationale

ReLU (Rectified Linear Unit)

- **Location**: After each GATConv (Line 308)
- **Why ReLU?**:
  - Sparse activation (zeros out negative values)
  - Computationally efficient
  - Prevents vanishing gradients in deep networks
  - Works well with attention mechanisms

No Activation in Final Layer

- **Design choice**: Classification head outputs raw logits
- **Why?**: `CrossEntropyLoss` applies softmax internally, avoiding double softmax

---

Regularization Techniques

Layer Normalization (Lines 215-217)

```
self.layer_norms = nn.ModuleList([
    nn.LayerNorm(hidden_dim) for _ in range(num_layers)
])
```

- **Why LayerNorm over BatchNorm?**: Graph data has variable batch sizes, LayerNorm works per-node
- **Placement**: After activation, before residual connection

Adaptive Dropout (Lines 232-235)

```
domain_dropout_rates = {
    'beth': 0.5,   # Higher for imbalanced data
    'unsw': dropout,  # Standard 0.3
    'optc': dropout,  # Balanced data
}
```

Technical Design Decisions

Why 2 GNN Layers?

- **Empirical testing** showed diminishing returns beyond 2 layers
- **Over-smoothing prevention**: More layers cause node embeddings to converge
- **Computational efficiency**: Balance between expressiveness and training time

Why 128 Hidden Dimension?

- **Power of 2**: Optimizes GPU memory alignment
- **Sufficient capacity**: Can encode complex attack patterns
- **Attention heads**: 4 × 32 = 128 provides diverse feature learning

Why Mean Aggregation?

```
self.convs.append(HeteroConv(conv_dict, aggr="mean"))
```

- **Robust to varying neighbor counts**
- **Preserves signal strength** across heterogeneous node types
- **Computationally efficient** compared to attention-based aggregation

---

Memory & Computational Complexity

Parameter Count Estimation

```
Input encoders: 9 × (input_dim × 128) ≈ 9K parameters
GAT layers: 2 × (128 × 128 × 4 heads) ≈ 131K parameters
Classification head: (128 × 64) + (64 × 10) ≈ 9K parameters
Total: ~150K parameters (very lightweight)
```

Memory Usage

- **Node features**: O(N × 128) where N = total nodes
- **Edge indices**: O(E × 2) where E = total edges
- **Attention weights**: O(E × heads × 4) for GAT

Computational Complexity

- **Message passing**: O(E × hidden_dim × heads)
- **Attention computation**: O(E × heads^2) per layer
- **Overall**: Linear in number of edges, scalable to large graphs

---

Summary

The MITRE-CORE HGNN architecture achieves the balance of **expressiveness**, **interpretability**, and **efficiency** required for real-time security correlation systems. The backbone freezing strategy enables effective transfer learning across domains while preserving universal attack pattern recognition capabilities.

Key Strengths:

- Heterogeneous graph modeling for multi-domain security data
- Attention mechanisms for interpretability
- Residual connections prevent over-smoothing
- Backbone freezing enables efficient domain adaptation
- Lightweight architecture suitable for real-time deployment

Design Philosophy:

- **Modularity**: Separate backbone and classification heads
- **Transferability**: Shared backbone across domains
- **Interpretability**: Attention weights for analyst trust
- **Efficiency**: Optimized for real-time SIEM integration